LANGGRAPH STATEGRAPH WORKFLOWS

Introduction

This document presents five practical workflow implementations using **LangGraph's StateGraph** in Python. The objective is simple:

- Two workflows are fully functional and execute without errors
- Three workflows intentionally contain realistic bugs

Each faulty workflow includes a corrected version along with explanations to help understand the root cause of the issues.

This material is designed both as a **learning resource** and a **debugging practice guide**, especially for those exploring graph-based orchestration in LLM pipelines.

About LangGraph

LangGraph is an extension of LangChain that enables developers to build **multi-step AI workflows using graph structures**.

- Nodes → represent individual processing steps
- Edges → define execution flow
- Conditional edges → allow branching based on runtime decisions

It is particularly useful for workflows involving **loops, retries, and dynamic routing logic**.

Structure of the Document

- **Section 1:** Functional Workflow A — Linear pipeline
- **Section 2:** Functional Workflow B — Conditional branching
- **Section 3:** Faulty Workflow C — Data pipeline errors + fixes
- **Section 4:** Faulty Workflow D — Routing and state issues + fixes
- **Section 5:** Faulty Workflow E — Looping and missing nodes + fixes

---

Dependencies

All examples rely only on:

- `langgraph`
- `typing_extensions` (for TypedDict)

Install using:

pip install langgraph

Python 3.9+ is recommended.

Section 1 — Workflow A (Functional)

Overview

A basic linear workflow consisting of two nodes. It demonstrates a simple input → processing → output pattern without branching.

Workflow Details

- **State Schema:** `message`, `result`
- **Step 1:** Input formatting
- **Step 2:** Response generation
- **Flow:** format → generate → END

Execution

Run the script:

python workflow_a.py

Expected Output:

{'message': 'hello world', 'result': 'Processed: hello world'}

Section 2 — Workflow B (Functional)

Overview

This workflow introduces **conditional routing** based on validation logic.

Workflow Details

- **State Fields:** `input`, `valid`, `output`
- **Steps:**
  - Validate input
  - Route to processing or error handler
- **Routing Logic:**
  - Valid → process node
  - Invalid → error node

Execution

python workflow_b.py

Sample Outputs:

Valid input: {'input': 'Hello', 'valid': True, 'output': 'Result: HELLO'}
Short input: {'input': 'Hi', 'valid': False, 'output': 'Error: input too short or empty'}

Section 3 — Workflow C (Faulty)

Overview

A simple ETL-style pipeline (load → enrich). This version contains three deliberate mistakes.

Identified Issues

Bug	Type	Description

1	Incorrect return type	Node returns a list instead of dict
2	Invalid entry point	Entry node name mismatch
3	Missing termination	Edge points to undefined node

Corrections

- Ensure all nodes return a **state dictionary**
- Fix entry point to match actual node name
- Replace invalid edge with `END`

Key Insight

LangGraph strictly requires **state consistency and valid graph termination paths**.

Section 4 — Workflow D (Faulty)

Overview

This workflow simulates a ticket handling system with priority-based routing.

Issues Found

Bug	Type	Description
1	Routing mismatch	Router returns value not mapped
2	Dead node	Node exists but is never used
3	Key mismatch	Incorrect state key reference

Fixes Applied

- Add missing routing condition (`normal`)
- Connect unused node into execution flow
- Correct state key usage

Key Insight

All routing outputs must be mapped, and **unused nodes should not exist in production workflows**.

Section 5 — Workflow E (Faulty)

Overview

An email processing pipeline with retry logic. Contains subtle but critical issues.

Issues Found

Bug	Type	Description
1	Infinite loop	Router never exits
2	Missing node	Referenced but not defined
3	Wrong key	Incorrect state field used

Fixes Applied

- Introduced retry limit to break loop
- Defined and registered missing node
- Corrected state key usage

Key Insight

Always ensure **exit conditions exist in loops** and all referenced nodes are defined.

Quick Summary of Bugs & Fixes

Workflow	Issue	Fix
C	Wrong return type	Return dict instead of list
C	Entry point mismatch	Use correct node name
C	No END path	Add proper termination
D	Missing router key	Add all possible cases
D	Unused node	Connect node properly
D	Key error	Fix state field name
E	Infinite loop	Add exit condition
E	Missing node	Define and register node
E	Wrong key	Use correct state field

Best Practices to Avoid Errors

1. Always return a **dictionary matching the state schema**
2. Ensure the **entry point matches a defined node**
3. Every workflow path must lead to **END**
4. Router outputs must match **mapping keys exactly**
5. All referenced nodes must be **registered beforehand**
6. Maintain **consistent state key names**
7. Avoid infinite loops by defining **clear exit conditions**

Conclusion

These examples highlight common mistakes developers encounter when building LangGraph workflows. While the functional workflows demonstrate correct patterns, the faulty ones serve as practical debugging exercises.

Understanding these patterns will help in designing **robust, scalable, and maintainable AI workflows** using LangGraph.